

Command

Padrões de Projeto Comportamental II

Prof. Me Jefferson Passerini



O padrão **Command** encapsula uma solicitação como um objeto, isso lhe permite parametrizar clientes com diferentes solicitações, enfileirar ou registrar (log) solicitações e suportar solicitações que podem ser desfeitas.

Aplicabilidade

- **Quando é preciso parametrizar objetos como ação a ser executada:**
 - o Tal parametrização pode ser expressada numa linguagem procedural através de uma função callback, ou seja, uma função que é registrada em algum lugar para ser chamada em um momento mais adiante.
 - o Os Comandos são uma substituição orientada a objetos para callbacks.

Aplicabilidade

- **Quando há necessidade de especificar, enfileirar e executar solicitações em tempos diferentes:**
 - Um objeto Comando, sabe quais ações executar e conhece o objeto receptor capaz de executá-las. Ele pode executar as ações a qualquer momento, deste modo, pode ter um tempo de vida independente da solicitação original.
 - É possível enfileirar Comandos para serem executados de forma controlada.

Aplicabilidade

- **Para suportar desfazer operações:**

- Para desfazer seus efeitos, um comando pode armazenar os estados em que o objeto receptor se encontrava antes da execução das ações (chamada do método `execute()`).
- A interface `Command` pode conter uma operação `undo()`, que reverte os efeitos de uma chamada anterior de `execute()`. De acordo com a complexidade da reversão ela pode ser baseada nos estados em que o receptor se encontrava anteriormente.
- Os comandos executados podem ser armazenados em uma lista histórica.
- O nível ilimitado de desfazer e refazer operações é obtido percorrendo esta lista para trás e para frente, chamando operações `undo()` e `execute()` respectivamente.

Aplicabilidade

- **Para tornar possível que mudanças no sistema sejam recuperadas por meio de um registro de mudanças (logging) em caso de queda:**
 - Ao aumentar a interface de Command com as operações load() e store(), pode-se manter um registro (log) persistente das mudanças feitas no objeto receptor.
 - A recuperação de uma queda de sistema envolve a recarga dos comandos registrados a partir do disco e sua reexecução com a operação execute().

Aplicabilidade

- **Se faz necessário estruturar um sistema em torno de operações de alto nível construídas sobre operações primitivas:**
 - Tal estrutura é comum em sistemas de informação que suportam transações.
 - Uma transação encapsula um conjunto de mudanças nos dados.
 - O padrão Command fornece uma maneira de modelar transações, já que os comandos podem ser desfeitos para o caso algum deles falhar.
 - Os Commands têm uma interface comum, permitindo invocar todas as transações da mesma maneira.
 - O padrão também torna mais fácil estender o sistema com novas transações.

Componentes

- **Ciente:** É o responsável pela criação de um ComandoConcreto e pela definição de seu receptor.
- **Invocador:** Contém um comando e em algum momento pede ao comando para entender uma solicitação chamando o seu método execute().

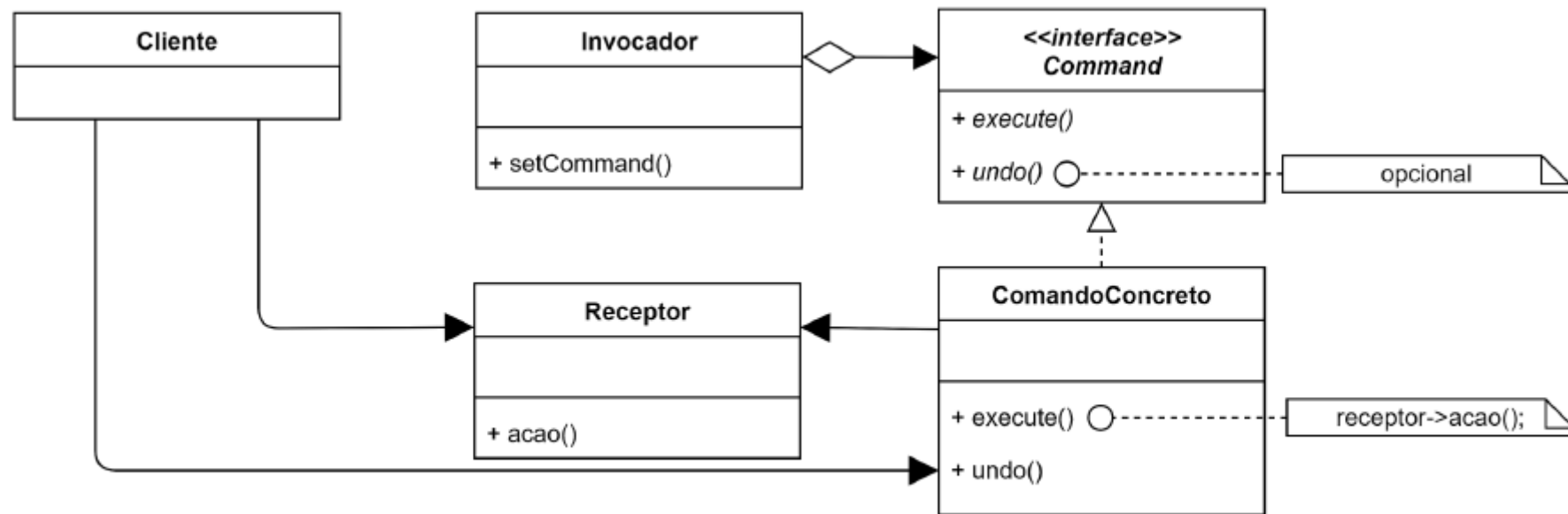


Diagrama de Classes

Componentes

- **Command:** Declara uma interface para todos os comandos. Um comando é invocado através de seu método `execute()`, que pede a um receptor para executar uma ação. Essa interface também pode possuir um método `undo()`, que desfaz a última ação executada.

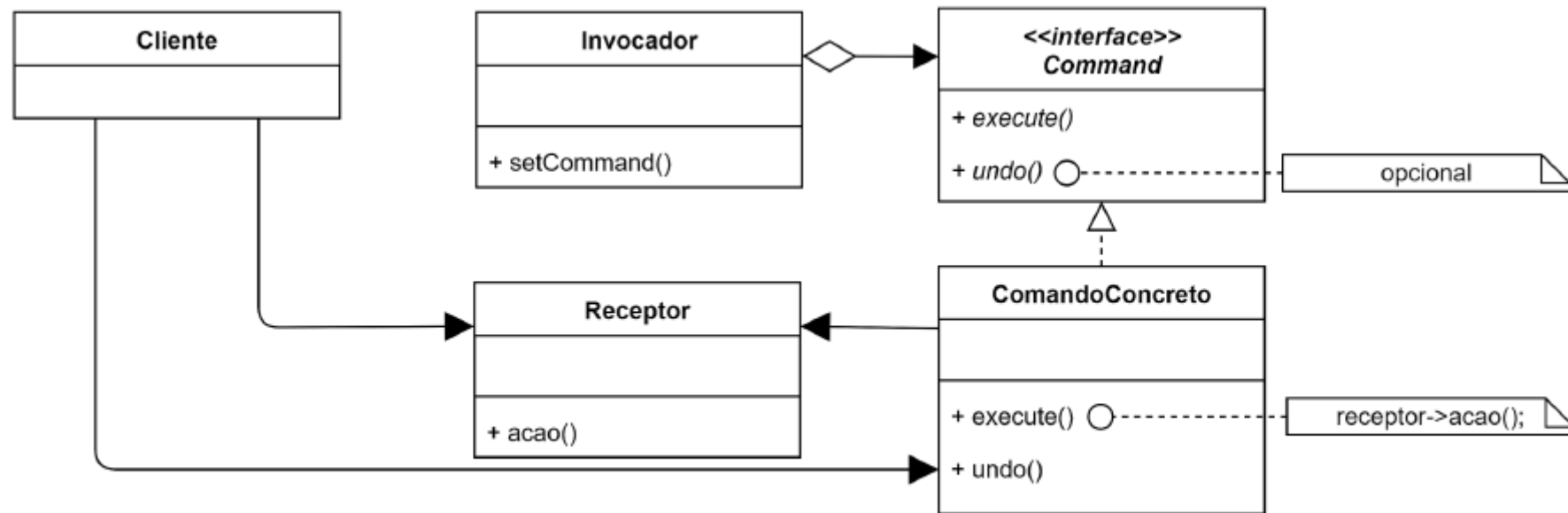


Diagrama de Classes

Componentes

- **ComandoConcreto:** Implementa a interface Command e define um vínculo entre uma ação e um objeto Receptor. Implementa o método execute através da invocação da(s) correspondente(s) operação(ões) no Receptor.

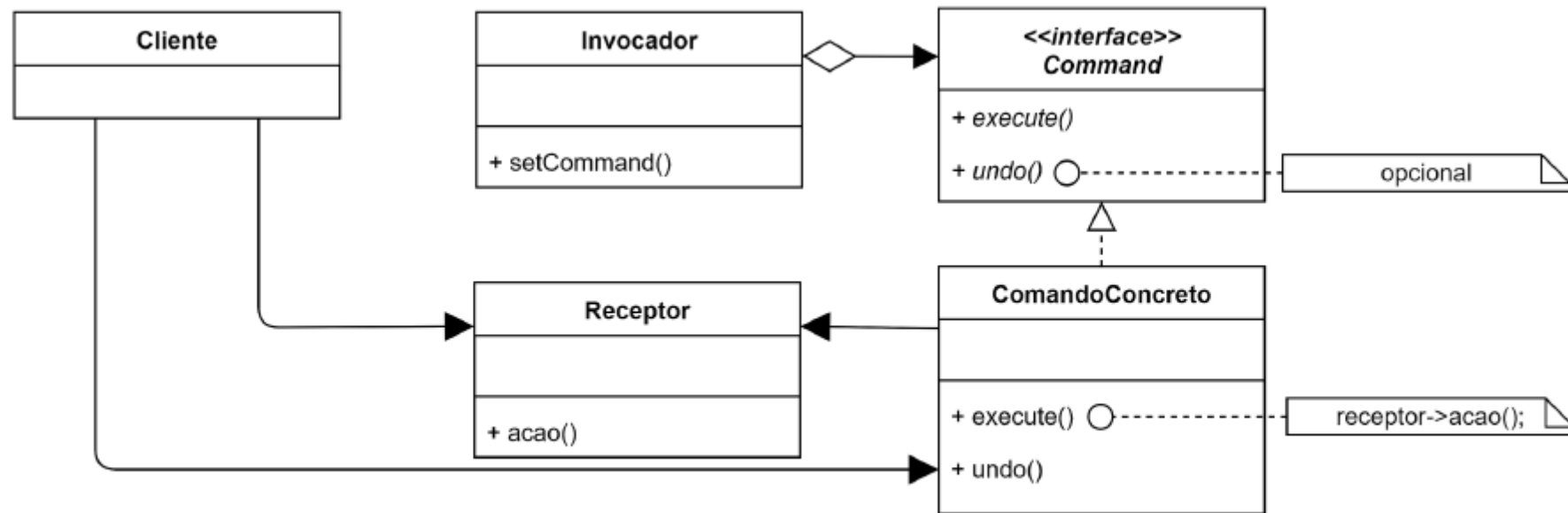


Diagrama de Classes

Componentes

- **Receptor:** O receptor sabe como executar as tarefas necessárias para atender a uma solicitação, qualquer classe pode atuar como um receptor.

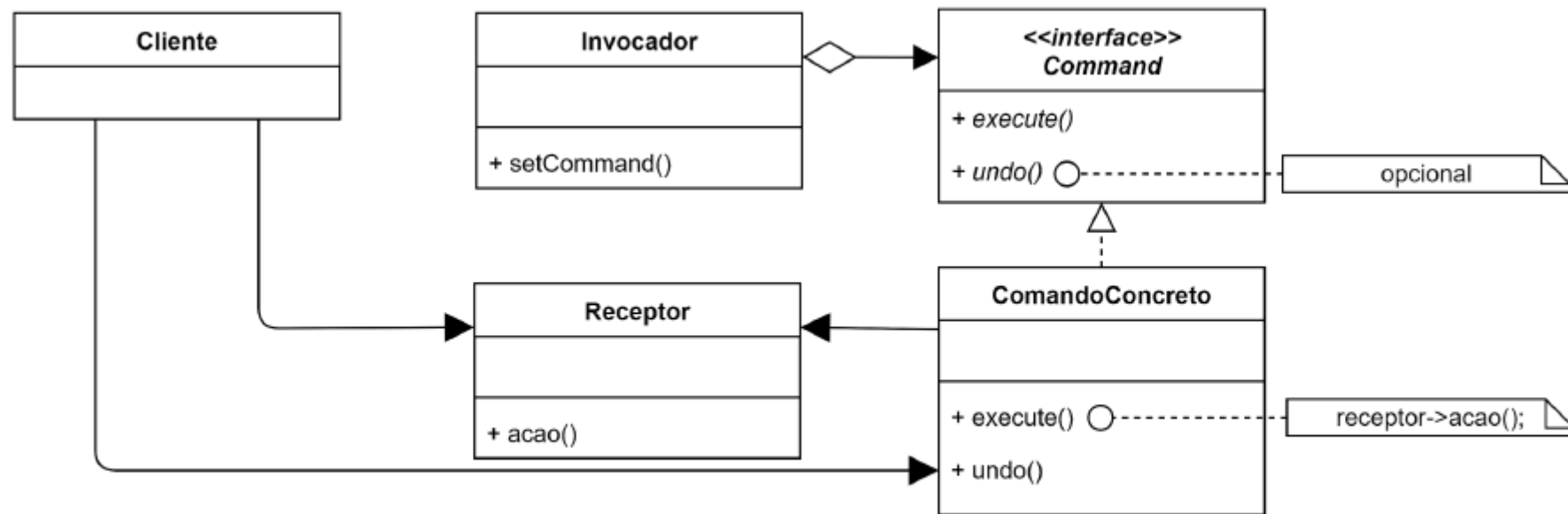
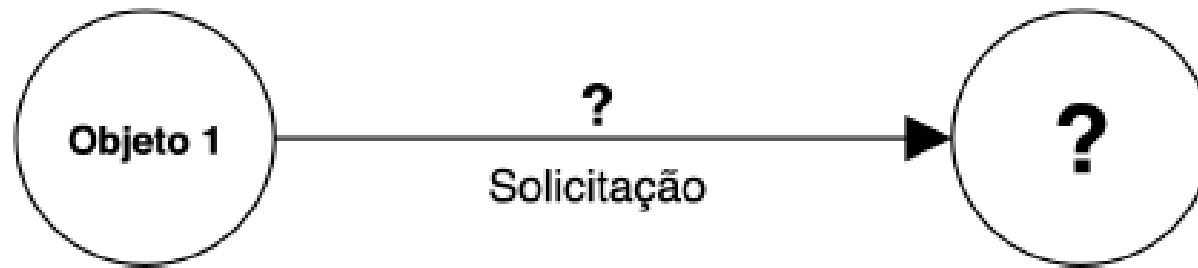


Diagrama de Classes

Motivação (Por que utilizar?)

- Considere um cenário onde é necessário emitir diferentes solicitações para diferentes objetos sem conhecer detalhes da operação que está sendo solicitada ou o destinatário da solicitação.
- E que ainda seja possível que, ao longo do tempo, novas solicitações possam ser suportadas pelo sistema e por isso ele deve ser expansível.



Solicitação desconhecida a um objeto desconhecido

Motivação (Por que utilizar?)

- É difícil pensar em um cenário como esse, porém, é nesse cenário que o Command se aplica muito bem.
- Vamos utilizar um exemplo para ilustrar um desses cenários. Imagine que precisamos implementar um aplicativo para controlar os dispositivos de uma casa pelo celular (internet of things).
- A casa possui 3 dispositivos inteligentes:
 - Lâmpada da sala, lâmpada do quarto, ar-condicionado;

Motivação (Por que utilizar?)

- Os proprietários da casa já avisaram que desejam adquirir novos produtos inteligentes no futuro, então nosso aplicativo deve ser facilmente expansível.
- Cada um dos dispositivos inteligentes aceita um conjunto específico de solicitações, suas classes já foram escritas e fornecidas pelos fabricantes dos dispositivos.

Command – Implementação C#

Padrões de Projeto Comportamental II

Prof. Me Jefferson Passerini



Motivação (Por que utilizar?)

- Os proprietários da casa já avisaram que desejam adquirir novos produtos inteligentes no futuro, então nosso aplicativo deve ser facilmente expansível.

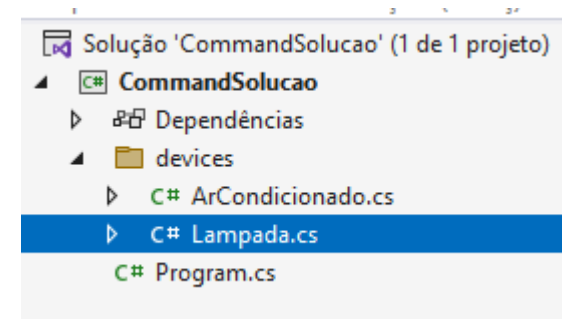
Lampada
- identificacao: string - estado: bool
+ ligar(): void + desligar(): void + getIdentificacao(): string + getEstado(): bool + imprimeObjeto(): void

ArCondicionado
- identificacao: string - estado: bool - temperatura: int
+ ligar(): void + desligar(): void + setTemperatura(int temperatura): void + getIdentificacao(): string + getEstado(): bool + getTemperatura(): int + imprimeObjeto(): void

Classes fornecidas pelos fabricantes dos dispositivos

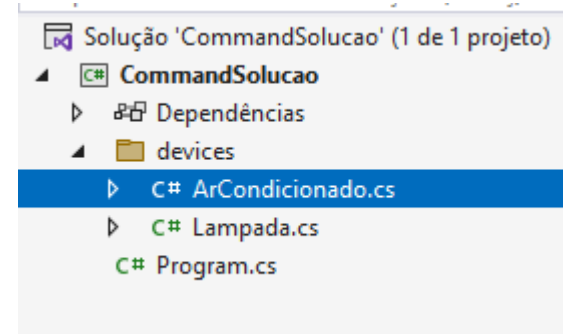
```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace CommandSolucao.devices
8  {
9      1 referência
10     public class Lampada
11     {
12         2 referências
13         public String identificacao { get; set; }
14         5 referências
15         public Boolean estado { get; set; }
16
17         0 referências
18         public Lampada(string identificacao, bool estado)
19         {
20             this.identificacao = identificacao;
21             this.estado = estado;
22         }
23
24         0 referências
25         public void ligar()
26         {
27             this.estado = true;
28         }
29
30         0 referências
31         public void desligar()
32         {
33             this.estado = false;
34         }
35
36         0 referências
37         public String getIdentificacao()
38         {
39             return this.identificacao;
40         }
41
42         0 referências
43         public Boolean getEstado()
44         {
45             return this.estado;
46         }
47     }
48 }
```

```
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```



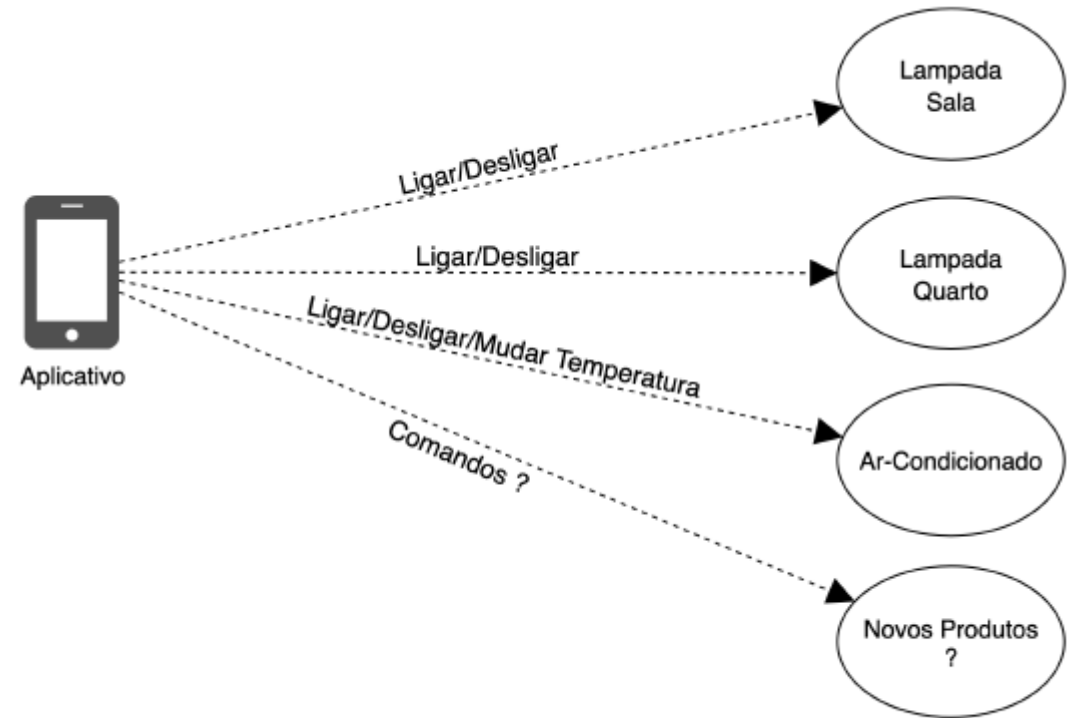
```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace CommandSolucao.devices
8 {
9     1 referência
10     public class ArCondicionado
11     {
12         2 referências
13         public String identificacao { get; set; }
14         5 referências
15         public Boolean estado { get; set; }
16         4 referências
17         public int temperatura { get; set; }
18
19         0 referências
20         public ArCondicionado(string identificacao, bool estado, int temperatura)
21         {
22             this.identificacao = identificacao;
23             this.estado = estado;
24             this.temperatura = temperatura;
25         }
26
27         0 referências
28         public void ligar()
29         {
30             this.estado = true;
31         }
32
33         0 referências
34         public void desligar()
35         {
36             this.estado = false;
37         }
38
39         0 referências
40         public void setTemperatura(int temperatura)
41         {
42             this.temperatura = temperatura;
43         }
44
45         0 referências
46         public String getIdentificacao()
47         {
48             return this.identificacao;
49         }
50     }
51 }
```

```
42
43
44     0 referências
45     public Boolean getEstado()
46     {
47         return this.estado;
48     }
49
50     0 referências
51     public int getTemperatura()
52     {
53         return this.temperatura;
54     }
55
56     0 referências
57     public String toString()
58     {
59         String texto = $"0 dispositivo esta {(this.estado ? "ligado" : "desligado")} a {this.temperatura} graus celsius";
60         return texto;
61     }
62 }
```



Motivação (Por que utilizar?)

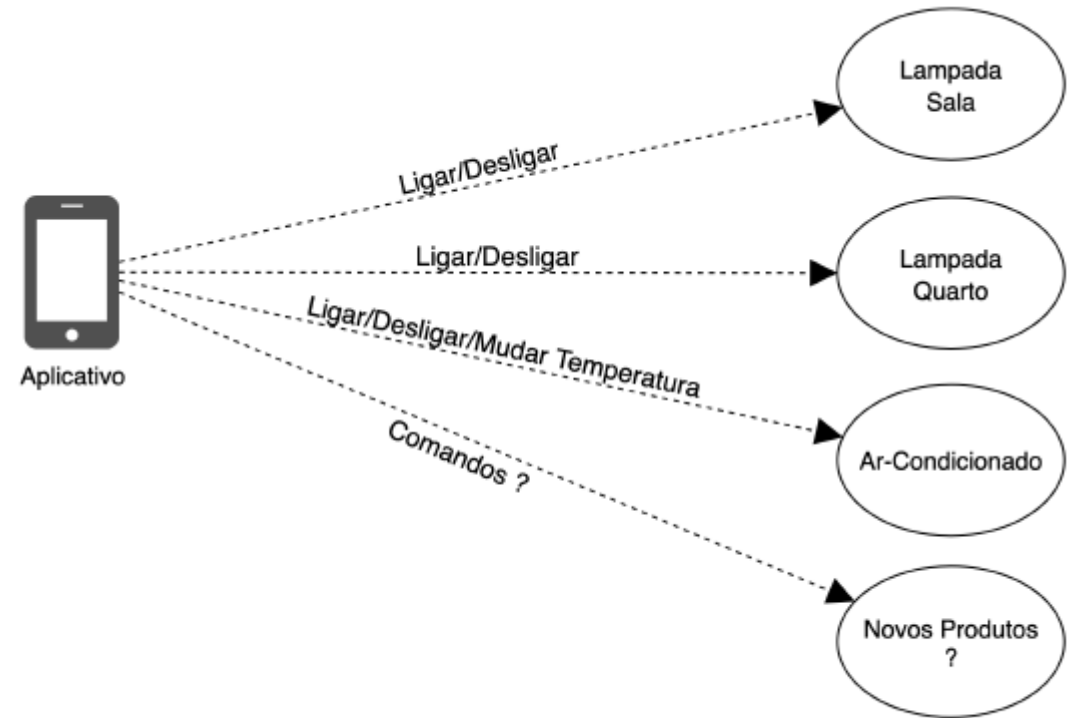
- Uma **Lampada** pode ser ligada e desligada, Um **ArCondicionado** também pode ser ligado e desligado, porém, também pode ter sua temperatura alterada.
- A casa inteligente deverá ser controlada por um aplicativo para smartphone e os comandos podem ser enviados para os respectivos objetos conforme a imagem a seguir.



Possíveis solicitações para possíveis objetos

Motivação (Por que utilizar?)

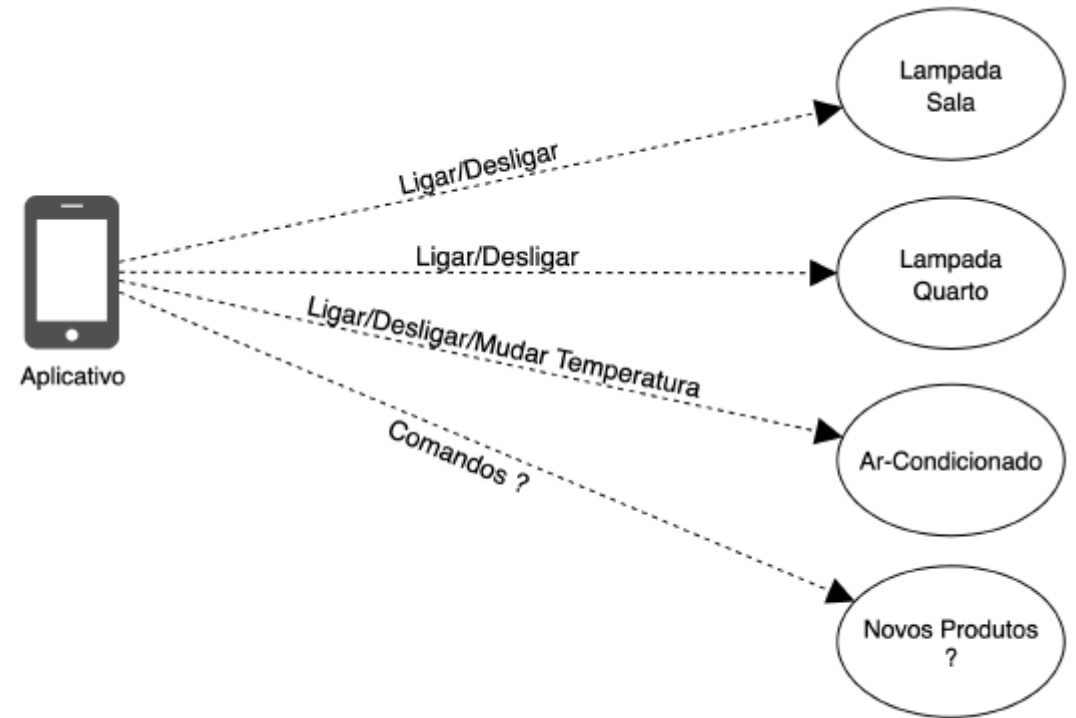
- Nosso objetivo é **centralizar** o controle de todos os objetos em um único lugar, no caso o Aplicativo.
- Os comandos devem ser enviados para os respectivos objetos somente quando o usuário interagir com o aplicativo, clicando em um botão “Ligar Lâmpada do Quarto” ou “Desligar Ar-condicionado” por exemplo.



Possíveis solicitações para possíveis objetos

Motivação (Por que utilizar?)

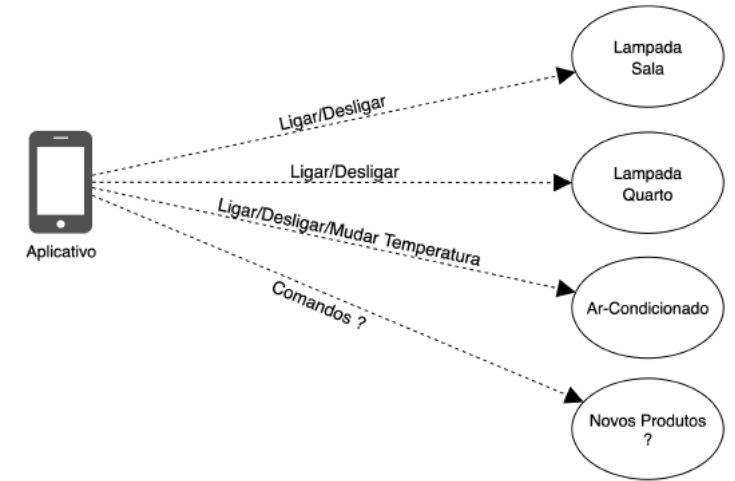
- Como podemos deixar os comandos pré-configurados no aplicativo de modo que possam ser invocados pelos proprietários quando eles desejarem?
- E ainda como manter o aplicativo aberto para que receba novos comandos dos novos dispositivos que forem adquiridos pelos proprietários?



Possíveis solicitações para possíveis objetos

Motivação (Por que utilizar?)

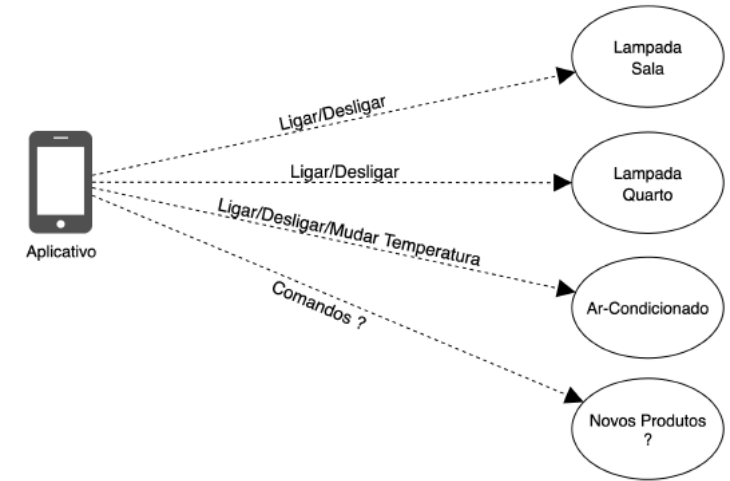
- O padrão de projeto **Command** sugere a criação de objetos de comando.
- Tais objetos encapsulam uma solicitação, vinculando um conjunto de ações a serem executadas em um objeto receptor específico.



Possíveis solicitações para possíveis objetos

Motivação (Por que utilizar?)

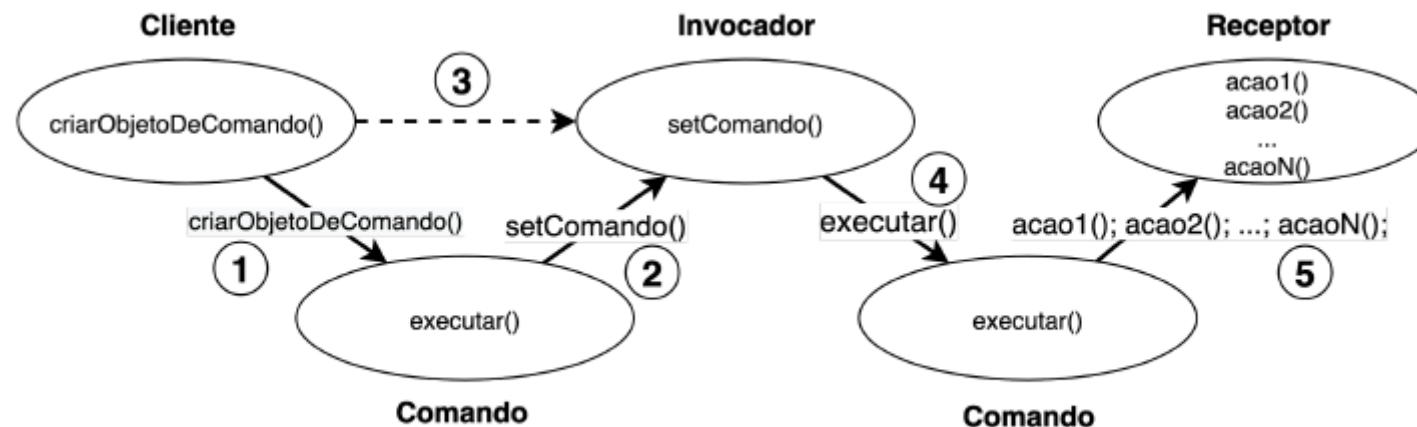
- Um objeto de comando tem as seguintes características:
 - Possui um conjunto de ações que devem ser executadas;
 - Conhece o receptor que deve executar as ações;
 - Expõe um método (`execute()` ou `executar()`) que quando chamado, executa as ações no objeto receptor.
 - Se necessário, pode expor um método (`undo()`, `desfazer()`) que desfaz as ações do método *executor*.



Possíveis solicitações para possíveis objetos

Motivação (Por que utilizar?)

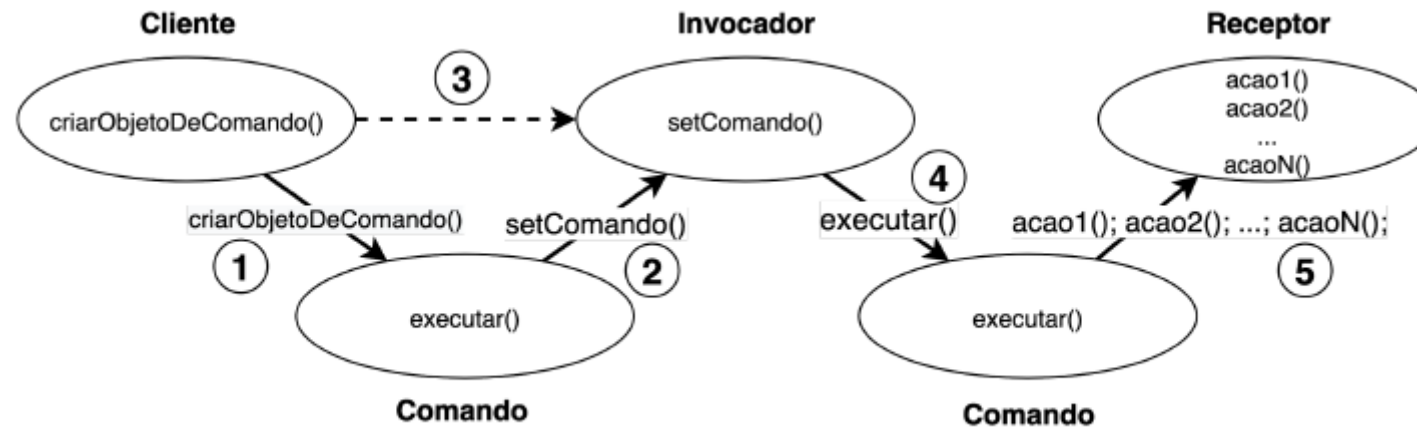
- ***Externamente nenhum objeto cliente sabe realmente quais ações são executadas em qual receptor.*** Eles sabem apenas que suas solicitações são atendidas quando o método executor é acionado.
- O fluxo de processamento desde a criação de um comando até a execução ocorre da seguinte maneira:



Fluxo de solicitação e execução de um comando

Motivação (Por que utilizar?)

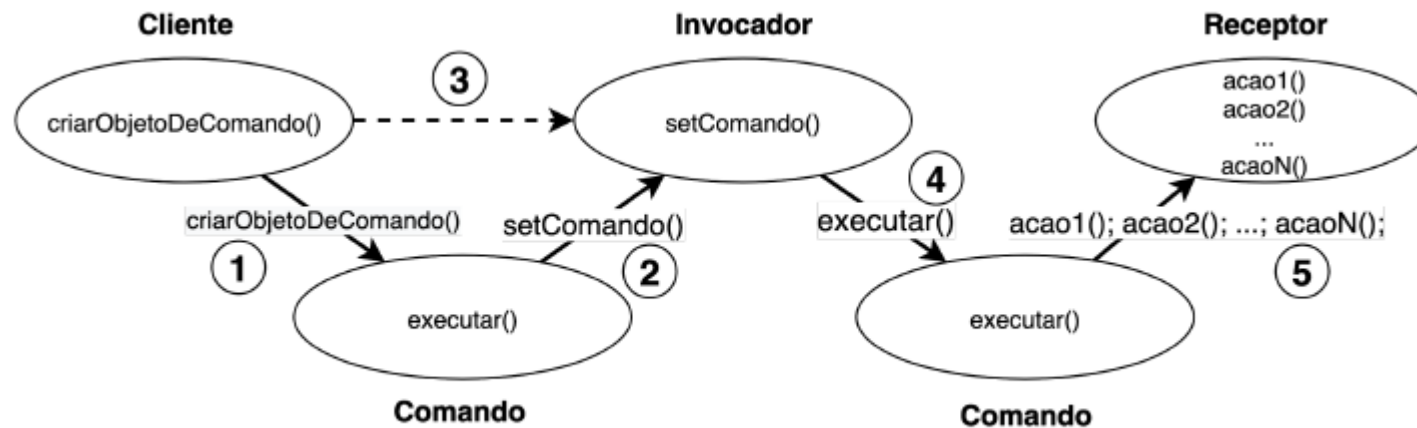
- **Cliente:** Responsável por criar o objeto de comando, que consiste em um conjunto de ações que serão executadas em um receptor.
- **Comando:** Conhece as ações que serão executadas e em qual objeto receptor serão executadas. Fornece um método **executar()**, que encapsula as ações e pode ser chamado para invocar tais ações no objeto receptor.
- **Invocador:** É o objeto intermediário entre um comando e seu receptor. O **Cliente** pede para o **invocador** chamar o método **executar()** do objeto de **Comando**. Possui o método **setComando()** que serve para o **Cliente** definir qual **Comando** o **invocador** deve executar.



Fluxo de solicitação e execução de um comando

Motivação (Por que utilizar?)

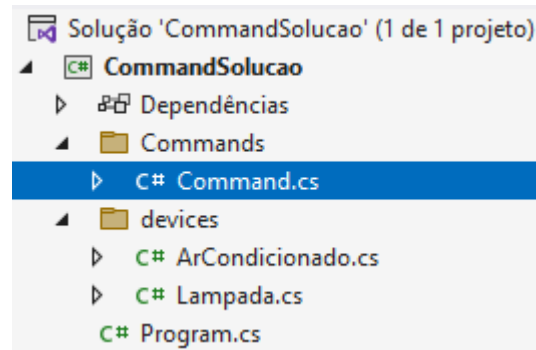
- **Receptor:** É o objeto que contém a implementação das ações, que por intermédio do **invocador**, serão chamadas pelo método **executar()** implementado no objeto de **Comando**.
 - 1. O cliente cria um objeto de **Comando**;
 - 2. O **Cliente** chama o método **setComando()** do **invocador** passando o objeto de **Comando** para ele. O objeto de **Comando** fica armazenado no **Invocador** esperando que seja acionado no futuro.
 - 3. No futuro o **Cliente** pede ao **Invocador** que chame o método **executar()** do **Comando**.
 - 4. O método **executar()** chama os métodos de **Acao()** do **Receptor**
 - 5. O Objeto **Receptor** os executa



Fluxo de solicitação e execução de um comando

Motivação (Por que utilizar?)

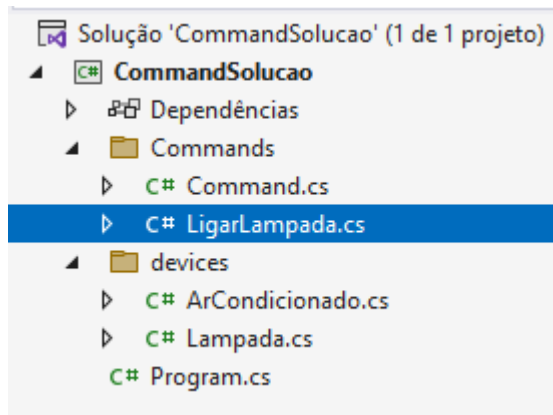
- Sabemos que todo comando precisa ter um método `executar()`. Também sabemos que um **Invocador** aceita apenas objetos do tipo **Comando**, então ele precisa ser capaz de verificar isso de alguma forma. No nosso exemplo também iremos implementar o método `desfazer()` de cada objeto de comando.
- Vamos criar uma interface (supertipo) para um comando.



```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace CommandSolucao.Commands
8  {
9      0 referências
10     public interface Command
11     {
12         0 referências
13         public void executar();
14         0 referências
15         public void desfazer();
16     }
17 }
```

Motivação (Por que utilizar?)

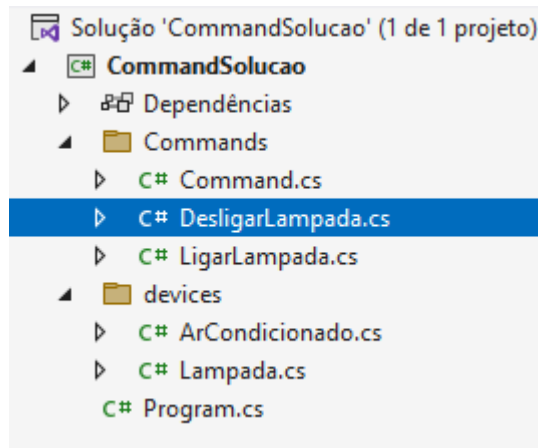
- Agora vamos identificar quais comandos serão necessários, todos devem implementar a interface Command.
- **LigarLampada**



```
1 using CommandSolucao.devices;
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5 using System.Text;
6 using System.Threading.Tasks;
7
8 namespace CommandSolucao.Commands
9 {
10     1 referência
11     internal class LigarLampada : Command
12     {
13         private Lampada lampada;
14
15         0 referências
16         public LigarLampada(Lampada lampada)
17         {
18             this.lampada = lampada;
19         }
20
21         1 referência
22         public void desfazer()
23         {
24             //no método desfazer, chamar o método desligar
25             this.lampada.desligar();
26             //chamar o método toString() para feedback
27             this.lampada.toString();
28         }
29
30         1 referência
31         public void executar()
32         {
33             //no método desfazer, chamar o método ligar
34             this.lampada.ligar();
35             //chamar o método toString() para feedback
36             this.lampada.toString();
37         }
38     }
39 }
```

Motivação (Por que utilizar?)

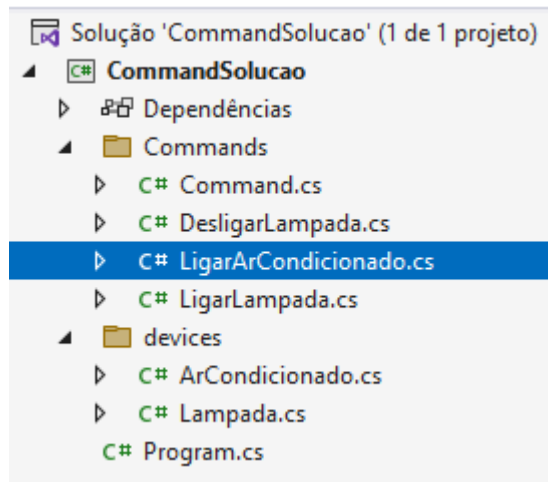
- Agora vamos identificar quais comandos serão necessários, todos devem implementar a interface Command.
- **DesligarLampada**



```
1 using CommandSolucao.devices;
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5 using System.Text;
6 using System.Threading.Tasks;
7
8 namespace CommandSolucao.Commands
9 {
10     1 referência
11     public class DesligarLampada : Command
12     {
13         private Lampada lampada;
14
15         0 referências
16         public DesligarLampada(Lampada lampada)
17         {
18             this.lampada = lampada;
19         }
20
21         1 referência
22         public void desfazer()
23         {
24             //no método desfazer, chamar o método ligar
25             this.lampada.ligar();
26             //chamar o método toString() para feedback
27             this.lampada.toString();
28         }
29
30         1 referência
31         public void executar()
32         {
33             //no método desfazer, chamar o método desligar
34             this.lampada.desligar();
35             //chamar o método toString() para feedback
36             this.lampada.toString();
37         }
38     }
39 }
```

Motivação (Por que utilizar?)

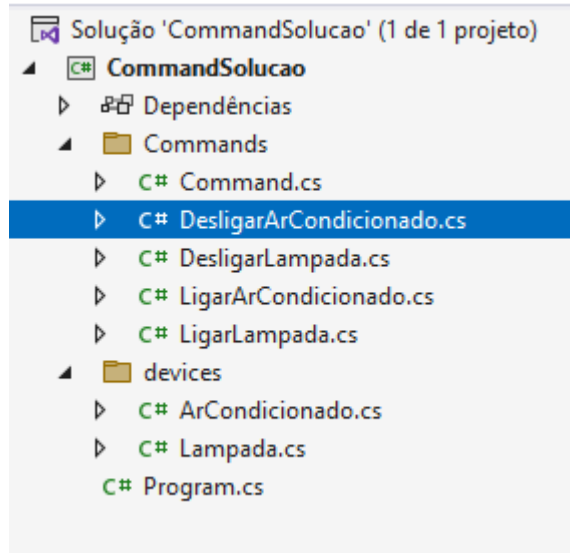
- Agora vamos identificar quais comandos serão necessários, todos devem implementar a interface Command.
- **LigarArCondicionado**



```
1 using CommandSolucao.devices;
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5 using System.Text;
6 using System.Threading.Tasks;
7
8 namespace CommandSolucao.Commands
9 {
10     1 referência
11     public class LigarArCondicionado : Command
12     {
13         private ArCondicionado arCondicionado;
14
15         0 referências
16         public LigarArCondicionado(ArCondicionado arCondicionado)
17         {
18             this.arCondicionado = arCondicionado;
19         }
20
21         1 referência
22         public void desfazer()
23         {
24             //no método desfazer, chamar o método desligar
25             this.arCondicionado.desligar();
26             //chamar o método toString() para feedback
27             this.arCondicionado.toString();
28         }
29
30         1 referência
31         public void executar()
32         {
33             //no método desfazer, chamar o método ligar
34             this.arCondicionado.ligar();
35             //chamar o método toString() para feedback
36             this.arCondicionado.toString();
37         }
38     }
39 }
```

Motivação (Por que utilizar?)

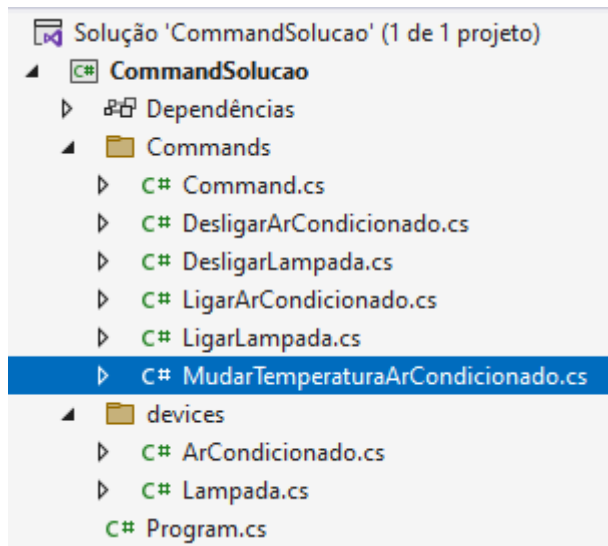
- Agora vamos identificar quais comandos serão necessários, todos devem implementar a interface Command.
- **DesligarArCondicionado**



```
1 using CommandSolucao.devices;
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5 using System.Text;
6 using System.Threading.Tasks;
7
8 namespace CommandSolucao.Commands
9 {
10     1 referência
11     public class DesligarArCondicionado : Command
12     {
13         private ArCondicionado arCondicionado;
14
15         0 referências
16         public DesligarArCondicionado(ArCondicionado arCondicionado)
17         {
18             this.arCondicionado = arCondicionado;
19         }
20
21         1 referência
22         public void desfazer()
23         {
24             //no método desfazer, chamar o método desligar
25             this.arCondicionado.ligar();
26             //chamar o método toString() para feedback
27             this.arCondicionado.toString();
28         }
29
30         1 referência
31         public void executar()
32         {
33             //no método desfazer, chamar o método ligar
34             this.arCondicionado.desligar();
35             //chamar o método toString() para feedback
36             this.arCondicionado.toString();
37         }
38     }
39 }
```

Motivação (Por que utilizar?)

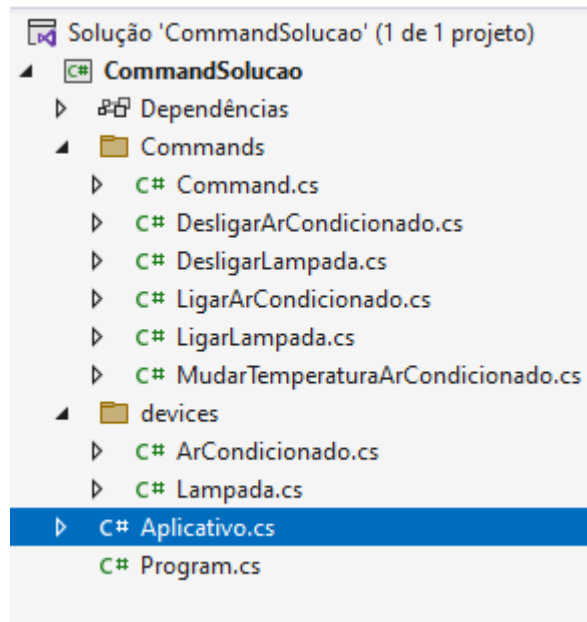
- Agora vamos identificar quais comandos serão necessários, todos devem implementar a interface Command.
- **MudarTemperaturaArCondicionado**



```
1 using CommandSolucao.devices;
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5 using System.Text;
6 using System.Threading.Tasks;
7
8 namespace CommandSolucao.Commands
9 {
10     1 referência
11     public class MudarTemperaturaArCondicionado : Command
12     {
13         private ArCondicionado arCondicionado;
14         private int temperatura;
15         private int temperaturaAnterior;
16
17         0 referências
18         public MudarTemperaturaArCondicionado(ArCondicionado arCondicionado)
19         {
20             this.arCondicionado = arCondicionado;
21             this.temperatura = arCondicionado.getTemperatura();
22         }
23
24         0 referências
25         public void setTemperatura(int temperatura)
26         {
27             this.temperaturaAnterior = this.temperatura;
28             this.temperatura = temperatura;
29         }
30
31         1 referência
32         public void desfazer()
33         {
34             //no método desfazer, seta a temp. anterior
35             this.arCondicionado.setTemperatura(this.temperaturaAnterior);
36             //chamar o método toString() para feedback
37             this.arCondicionado.toString();
38         }
39
40         1 referência
41         public void executar()
42         {
43             //no método executar, seta a temp.
44             this.arCondicionado.setTemperatura(this.temperatura);
45             //chamar o método toString() para feedback
46             this.arCondicionado.toString();
47         }
48     }
49 }
```

Motivação (Por que utilizar?)

- Temos os comandos devidamente implementados, então vamos implementar o nosso **Invocador**, que será o back-end do nosso aplicativo.
- **Aplicativo**



```
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5 using System.Text;
6 using System.Threading.Tasks;
7
8 namespace CommandSolucao
9 {
10     0 referências
11     public class Aplicativo
12     {
13         private List<Command> comandos = new List<Command>();
14
15         //para um comando ser adicionado a lista de comandos
16         //do app ele precisa implementar a interface command,
17         //assim sabemos que ele possui o método executar.
18         0 referências
19         public int setComando(Command comando)
20         {
21             //adiciona o comando na ultima posição
22             this.comandos.Add(comando);
23             //retorna para o cliente a posição do array em que foi
24             //inserido.
25             return comandos.Count-1;
26         }
27
28         //metodo que será chamado sempre sera q um botão for press.
29         //na interface do app. passando o indice a ser executado.
30         0 referências
31         public void apPrecionarBota(int id)
32         {
33             this.comandos[id].executar();
34         }
35
36         //metodo sera chamado sempre que o botão for pressionado
37         //duas vezes rapidamente na interface do app.
38         //precisa passar o indice do comando a ser desfeito
39         0 referências
40         public void duploCliqueBotao(int id)
41         {
42             this.comandos[id].desfazer();
43         }
44
45         //retorna um comando do app.
46         0 referências
47         public Command GetCommand(int id)
48         {
49             return this.comandos[id];
50         }
51     }
52 }
```

Explicação do fluxo da informação

- Para exemplificar vamos analisar passo a passo o fluxo de um comando que liga uma lâmpada (para simplificar desconsidere o método `desfazer()`).

- 1. O cliente cria um objeto de **Comando** no caso um objeto **LigarLampada** que recebe uma **Lampada** em seu construtor;

```
//Primeiro precisamos de um objeto do tipo Lâmpada
$Lampada = new Lampada('Lampada', false); //É inicializada desligada.

//Agora podemos criar o comando passando o objeto lâmpada em seu construtor;
$LigarLampada = new LigarLampada($Lampada);
```

- 2. O **Cliente** chama o método `setComando()` do invocador passando o objeto de **Comando** para ele. O objeto de **Comando** fica armazenado no **Invocador** esperando que seja acionado no futuro.

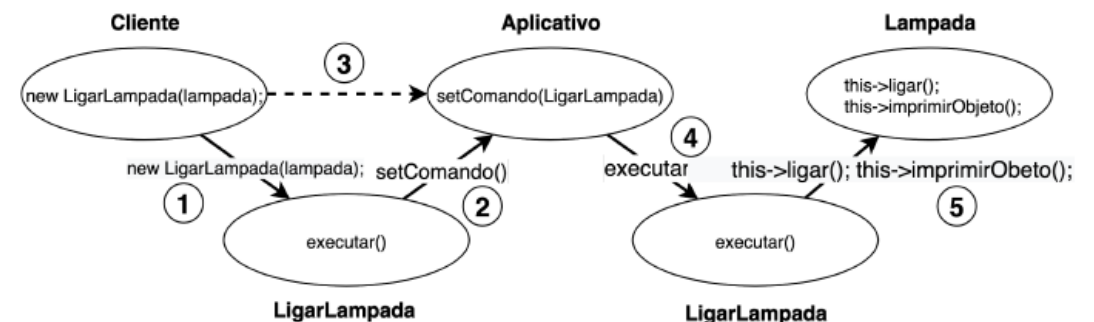
```
//Precisamos do invocador que é nosso aplicativo
$aplicativo = new Aplicativo();

//Agora podemos armazenamos o comando no aplicativo e guardamos o ID atribuído a ele.
$IdLigarLampada = $aplicativo->setComando($LigarLampada);
```

- 3. No futuro o **Cliente** pede ao **Invocador** que chame o método `executar()` do **Comando**. Nosso **Invocador** é o aplicativo.

```
//O proprietário clicou no botão que deve acender a lâmpada
//Nesse caso chamamos o método aoPrecionarBotao() do aplicativo passando o ID do comando.
$aplicativo->aoPrecionarBotao($IdLigarLampada);
```

- O método `aoPrecionarBotao()` chamará o método `executar()` do objeto de comando referente ao Id que passamos por parâmetro, que é o objeto **LigarLampada**.
- 4. O método `executar()` chama os métodos de `Acao()` do **Receptor**, que são os métodos `ligar()` e `imprimeObjeto()` do Objeto **lampada**.
- 5. Por fim, o objeto **Receptor** **lampada** os executa.



Fluxo de solicitação e execução do comando **LigarLampada**

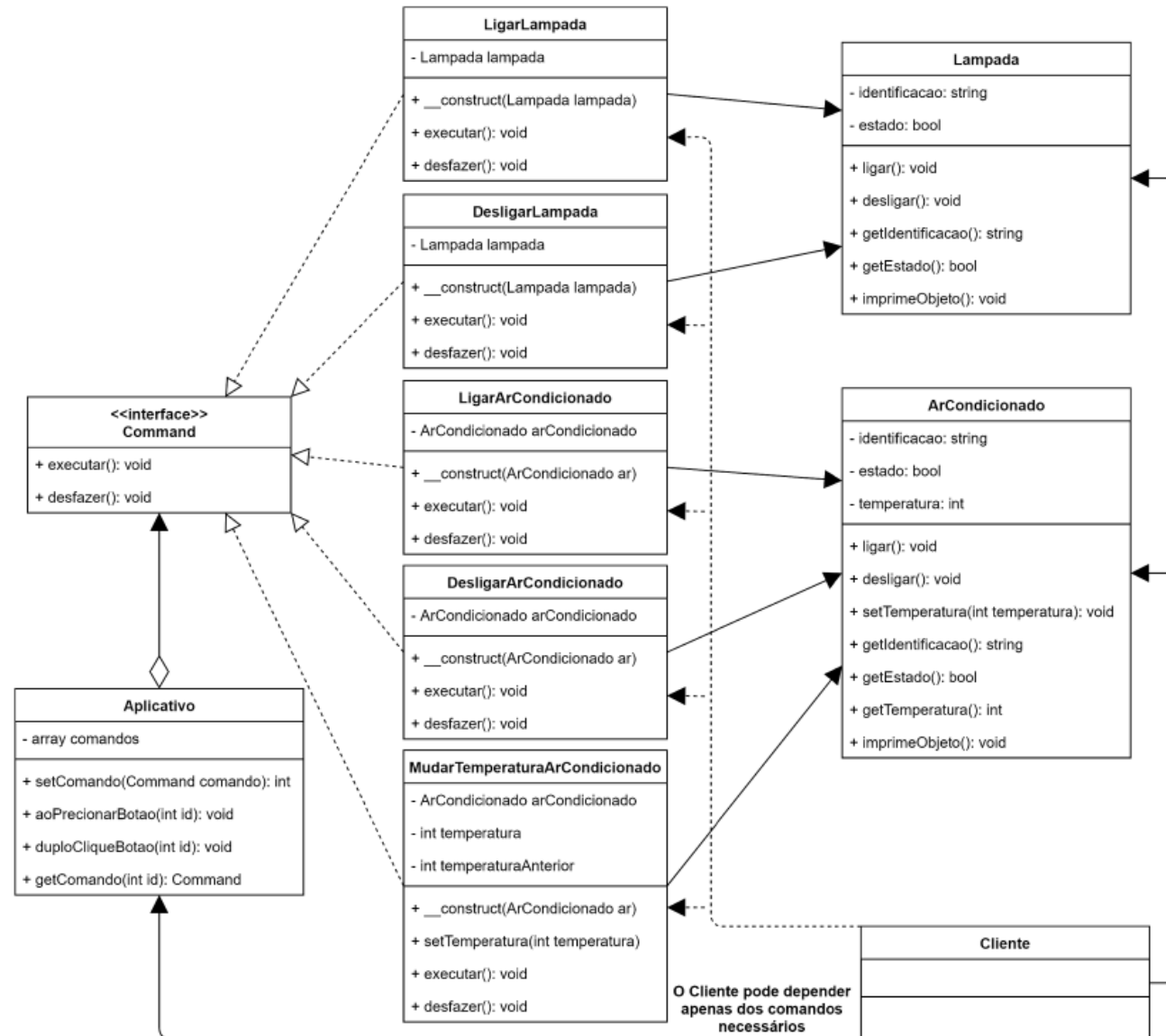
Implementando a classe Program.

```
1 using CommandSolucao;
2 using CommandSolucao.devices;
3 using CommandSolucao.Commands;
4
5 //criando dispositivos
6 Lampada lampadaSala = new Lampada("Lampada Sala", false);
7 Lampada lampadaQuarto = new Lampada("Lampada Quarto", false);
8 ArCondicionado ar = new ArCondicionado("Ar Condicionado", false, 26);
9
10 //criar app
11 Aplicativo meuApp = new Aplicativo();
12
13 //criar comandos
14 Command ligarLampadaSala = new LigarLampada(lampadaSala);
15 Command desligarLampadaSala = new DesligarLampada(lampadaSala);
16
17 Command ligarLampadaQuarto = new LigarLampada(lampadaQuarto);
18 Command desligarLampadaQuarto = new DesligarLampada(lampadaQuarto);
19
20 Command ligarAr = new LigarArCondicionado(ar);
21 Command desligarAr = new DesligarArCondicionado(ar);
22 Command mudarTemperatura = new MudarTemperaturaArCondicionado(ar);
23
24 //armazenar comandos no app
25 //os ids recebem o indice do comando adicionado no app
26
27 int idLigarLampadaSala = meuApp.setComando(ligarLampadaSala);
28 int idDesligarLampadaSala = meuApp.setComando(desligarLampadaSala);
29
30 int idLigarLampadaQuarto = meuApp.setComando(ligarLampadaQuarto);
31 int idDesligarLampadaQuarto = meuApp.setComando(desligarLampadaQuarto);
32
33 int idLigarAr = meuApp.setComando(ligarAr);
34 int idDesligarAr = meuApp.setComando(desligarAr);
35 int idMudarTemperatura = meuApp.setComando(mudarTemperatura);
36
37 //vamos executar os comandos.
38 meuApp.aoPrecionarBotao(idLigarLampadaSala);
39 meuApp.aoPrecionarBotao(idDesligarLampadaSala);
```

```
40
41 meuApp.aoPrecionarBotao(idLigarLampadaQuarto);
42 meuApp.aoPrecionarBotao(idDesligarLampadaQuarto);
43
44 meuApp.aoPrecionarBotao(idLigarAr);
45 //é necessário utilizar um Cast para transformar a interface Command na Classe Comando.
46 ((MudarTemperaturaArCondicionado)meuApp.getCommando(idMudarTemperatura)).setTemperatura(25);
47 meuApp.aoPrecionarBotao(idMudarTemperatura);
48 meuApp.aoPrecionarBotao(idDesligarAr);
49
50 //desfazer
51 meuApp.duploCliqueBotao(idDesligarAr);
52 meuApp.duploCliqueBotao(idMudarTemperatura);
53 meuApp.duploCliqueBotao(idLigarAr);
54 meuApp.duploCliqueBotao(idDesligarLampadaQuarto);
55
```

```
0 dispositivo Lampada Sala esta ligado
0 dispositivo Lampada Sala esta desligado
0 dispositivo Lampada Quarto esta ligado
0 dispositivo Lampada Quarto esta desligado
0 dispositivo esta ligado a 26 graus celsius
0 dispositivo esta ligado a 25 graus celsius
0 dispositivo esta desligado a 25 graus celsius
0 dispositivo esta ligado a 25 graus celsius
0 dispositivo esta ligado a 26 graus celsius
0 dispositivo esta desligado a 26 graus celsius
0 dispositivo Lampada Quarto esta ligado
```

Modelo Implementado



Command – Implementação Java

Padrões de Projeto Comportamental II

Prof. Me Jefferson Passerini



Command – Consequências

Padrões de Projeto Comportamental II

Prof. Me Jefferson Passerini



Consequências

- Comandos são objetos de primeira classe, ou seja, podem ser manipulados e estendidos como qualquer outro objeto.
- Um comando pode ser composto por outros comandos.
- É fácil acrescentar novos comandos porque não é preciso mudar as classes existentes.
- Implementar operações de desfazer fica mais fácil.
- O código pode se tornar mais complexo por existir camadas entre o cliente (solicitante) e o objeto receptor.